

Neural ODE vs. ResNet: An Empirical Comparison on Image Classification

Rogelio Lozano Joe Mota

Department of Computer Science, The University of Texas at El Paso

{rlozano12, jamota}@miners.utep.edu

1. Introduction

Modern deep learning has long relied on stacking discrete layers to learn complex representations. Residual Networks (ResNets) [2] pushed this idea further by introducing skip connections that ease gradient flow and allow much deeper models to train successfully. Still, ResNets operate at a fixed computational depth: you decide how many layers to stack at design time, and every input goes through all of them.

Neural Ordinary Differential Equations (Neural ODEs), introduced by Chen et al. [1], reframe this paradigm entirely. Instead of discrete layer-by-layer updates, a Neural ODE parameterizes the hidden state dynamics as a continuous ODE:

$$\frac{dh(t)}{dt} = f(h(t), t, \theta)$$

and uses a standard ODE solver to integrate from $t = 0$ to $t = 1$. The key insight is that ResNets can be interpreted as Euler discretizations of such a system, so Neural ODEs are essentially the continuous-depth generalization. Two notable advantages follow: (1) the ODE solver adaptively determines how many steps to take depending on the input complexity, and (2) the adjoint sensitivity method allows backpropagation through the solver in *constant* memory, independent of the number of solver steps.

The paper’s main claim is that Neural ODEs match ResNet accuracy while using significantly fewer parameters and less memory. Our project re-implements both architectures from scratch and evaluates them on MNIST and CIFAR-10 to assess how well this claim holds in practice.

2. Result & Analysis

We focused on Table 1 in Chen et al. [1], which reports that a Neural ODE classifier matches a comparable ResNet on MNIST while using only a fraction of the parameters and achieving lower peak memory usage via the adjoint method. Specifically, the paper demonstrates that continuous-depth models can be competitive with discrete-depth baselines without the fixed memory cost of storing all intermediate activations.

We targeted this result because it is the central empirical justification for Neural ODEs as a practical alternative to ResNets, not just a theoretical curiosity. If this claim holds, Neural ODEs become a compelling option in memory-constrained settings.

3. Methodology

Architectures. We implemented a small ResNet (`SmallResNet`) with three stages of residual blocks (conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ conv $\rightarrow$ BN, with skip connections), followed by global average pooling and a fully connected classifier. For the Neural ODE, we followed the paper’s design: a CNN downsampling stem reduces spatial resolution, an `ODEBlock` solves the continuous dynamics in the resulting feature space using `torchdiffeq` [3], and a pooling-plus-FC head produces class logits. The ODE function $f(t, h)$ is itself a small convolutional network. We used the adjoint method (`odeint_adjoint`) for memory-efficient backpropagation, matching the paper.

Datasets. MNIST [4] (70k grayscale 28×28 images, 10 classes) and CIFAR-10 [5] (60k RGB 32×32

images, 10 classes). Both were split into 55k/5k train/val and 10k test sets. CIFAR-10 training used standard light augmentation (random crop + horizontal flip).

Training. All models were trained with Adam [6] at a learning rate of 10^{-3} with cosine annealing. MNIST ran for 20 epochs and CIFAR-10 for 60 epochs. The ODE solver was `dopri5` (adaptive Runge-Kutta 4/5) with tolerances $\text{rtol} = \text{atol} = 10^{-2}$. We tracked test accuracy, total training time, peak GPU memory, and the number of function evaluations (NFE) per batch for the Neural ODE.

Parameter matching. We tuned base channel widths so the Neural ODE had approximately $3\times$ fewer parameters than the ResNet on both datasets, consistent with the paper’s setup.

Dataset	Model	Params	Test Acc.	Train Time	Peak Mem.
MNIST	ResNet	174,970	99.53%	120.5s	145 MB
MNIST	Neural ODE	52,138	96.85%	946.7s	121 MB
CIFAR-10	ResNet	696,618	90.84%	550.8s	520 MB
CIFAR-10	Neural ODE	207,818	65.44%	2962.6s	290 MB

Table 1: Summary of re-implementation results. Both Neural ODE models use $\sim 3.4\times$ fewer parameters and significantly less GPU memory than their ResNet counterparts.

4. Results & Analysis

MNIST. The ResNet hit 99.53% test accuracy in about 2 minutes. The Neural ODE reached 96.85%—only 2.7 points lower—with $3.4\times$ fewer parameters and 16% less peak memory, but at the cost of nearly $8\times$ longer training time due to the ODE solver overhead. This is broadly consistent with the original paper’s claim: the Neural ODE is competitive on a simpler dataset while being more parameter- and memory-efficient.

CIFAR-10. This is where things diverged from the paper’s claims. The ResNet achieved 90.84% test accuracy, while the Neural ODE only reached 65.44%—a 25-point gap that is much larger than anything reported or implied in the original work. Training the Neural ODE on CIFAR-10 also took nearly 50 minutes compared to about 9 minutes for ResNet. The NFE per batch averaged around 75 during training, meaning the adaptive solver was taking many more steps on the richer image distribution, which drove up both compute time and instability.

Looking at the val accuracy curves during CIFAR-10 training, the Neural ODE oscillated wildly between epochs—jumping from 62% down to 10-20%—which suggests the adaptive solver was struggling to maintain a stable solution trajectory as the loss landscape shifted throughout training. This kind of instability is less of an issue for simpler datasets like MNIST.

A few factors likely explain the gap: our ODE function is a shallow two-layer convolutional network operating in a fixed 64-channel feature space, whereas the ResNet has three progressively wider stages. The continuous-depth model simply may not have had enough representational power in the ODE dynamics for CIFAR-10’s complexity. Additionally, the tight solver tolerances (10^{-2}) we used for speed may have introduced approximation errors that hurt generalization on a harder task.

That said, the memory efficiency result held up clearly: the Neural ODE used 44% less GPU memory than the ResNet on CIFAR-10 (290 MB vs. 520 MB), with $3.4\times$ fewer parameters. So the efficiency story is real—the accuracy cost is just much larger than we expected.

5. Reflections

The biggest lesson for us was how much harder it is to train Neural ODEs on complex data compared to simple benchmarks. The adaptive solver introduces a feedback loop between the loss landscape and the number of compute steps taken—as training progresses and the ODE dynamics change, the solver adjusts its step count, which leads to variable training times and potential instability. This is very different from a ResNet, where every epoch is essentially identical in structure.

We also underestimated how sensitive the Neural ODE is to solver tolerances. Tighter tolerances would likely have improved accuracy but at a significant speed cost, and we were already at 50 minutes per CIFAR-10 run. Looser tolerances sped things up but seemed to hurt stability. In hindsight, experimenting with fixed-step solvers (like Euler or RK4) would have given us more control over this tradeoff.

Going forward, the most natural next steps would be: (1) scaling the ODE feature extractor—using a deeper or wider CNN feature space—to give the dynamics more expressiveness on harder tasks; (2) testing hybrid architectures that combine ResNet stages for feature extraction with a single ODE block for the final processing; and (3) experimenting with fixed-step solvers to directly control the speed/accuracy tradeoff. Benchmarking on CIFAR-100 or TinyImageNet would also be useful to better characterize where continuous-depth models break down.

Overall, this project gave us a much deeper appreciation for what Neural ODEs actually do versus what they sound like on paper. The continuous-depth framing is elegant and the memory efficiency is real, but matching ResNet performance on anything beyond simple datasets requires careful architecture choices and training stability work that the original paper understandably glosses over.

References

- [1] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural ordinary differential equations,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [3] R. T. Q. Chen, “torchdiffeq: Differentiable ODE solvers with full GPU support and adjoint backpropagation,” GitHub, 2018. <https://github.com/rtqichen/torchdiffeq>
- [4] Y. LeCun, C. Cortes, and C. J. Burges, “The MNIST database of handwritten digits,” 1998.
- [5] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.
- [6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
- [7] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.